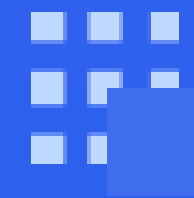




软件测试

简单随机测试



随机测试概述



随机测试是一种最简单的多样性测试方法。测试数据通过随机抽样生成，通常基于概率分布生成，也可用于模仿用户对软件的使用场景。

随机测试

随机测试能够低成本生成大量的测试，以发现隐藏的缺陷。随机测试可以用于不同的测试场景，具有很强的适用性和很高的易用性，广泛应用于各类测试。

优点

随机测试的缺陷检测效率瓶颈问题严重，随着测试进行，效率提升困难。后期能力提升困难，结合程序分析和其他测试方法改进是主要研究方向。

缺点

随机测试类型与应用场景



结合其他技术

通常与其他测试技术结合使用，如与白盒测试结合，实现更全面测试。例如，先用白盒测试覆盖代码逻辑，再用随机测试发现潜在问题。

随机测试类型

分为简单随机测试、自适应随机测试、引导性随机测试。不同分类方法适用于不同测试需求和场景。

随机测试的定义

01

定义

针对输入空间 Ω ，随机测试以概率分布 F 进行随机抽样获得测试集 T 。

- 默认情况下 F 是均匀分布。
- 实际应用中通常采用有放回抽样以简化实现。

02

基本思想

生成大量随机输入，记录程序输出，分析测试结果。输入可以是数据输入，界面交互输入等。

简单随机测试方法

- 示例：三角形程序
- 输入：随机生成三个整数 a,b,c（范围 1 到 100）
- 预期结果：根据三角形的性质计算。
- 实际结果：运行程序，比较结果

```
for i in range(num_tests):  
    a = random.randint(1, 100)  
    b = random.randint(1, 100)  
    c = random.randint(1, 100)  
    print("Test case {}: a={}, b={}, c={}".format(i+1, a, b, c))
```


登录系统的随机测试策略



目标：测试用户名为邮箱、密码的登录系统



步骤1：生成随机邮箱

使用faker库：

```
from faker import Faker
fake = Faker()
email = fake.email()
```

使用正则表达式：

```
import random
def generate_random_email():
    pattern = r"^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$"
    email = ""
    while not email:
        candidate = "user_" + str(random.randint(100, 999)) +
"@example.com"
        if re.match(pattern, candidate):
            email = candidate
    return email
```

登录系统的随机测试策略（续）



目标：包含大小写字母和数字，长度 ≥ 8 位



步骤2：随机生成密码

```
import random
import string
def generate_password(length):
    letters = string.ascii_letters
    digits = string.digits
    password = ''.join(random.choice(letters + digits) for i in range(length))
    return password
password = generate_password(8)
```

步骤3：模拟登录测试：将生成的邮箱和密码作为输入，模拟用户登录操作

人脸识别系统的随机测试策略



目标：测试人脸识别系统的鲁棒性和准确性



策略1：生成随机人脸图形

使用 OpenCV 库生成随机人脸图像

```
import random
face_image = ['clear', 'blurry', 'low quality']
resolution = ['low', 'medium', 'high']
lighting = ['bright', 'dim']
angle = ['front', 'side', 'top']
for i in range(100):
    face = random.choice(face_image)
    res = random.choice(resolution)
    light = random.choice(lighting)
    ang = random.choice(angle)
    print(f"Test case {i+1}: {face}, {res}, {light}, {ang}")
```


随机测试的高级策略

- 生成随机人脸图像：可以使用Python中的OpenCV库生成随机的人脸图像。这些图像可以包括不同年龄、性别、肤色、表情和光照等因素，以保证生成的测试用例覆盖尽可能多的情况。
- 模拟真实场景：可以使用真实场景下的图像或视频作为测试用例，例如从摄像头或社交媒体平台中获取图像。这些图像可以包括不同角度、距离、环境和光照等因素，以模拟真实的使用场景。
- 添加噪声和干扰：可以在图像中添加不同类型的噪声和干扰，例如高斯噪声、运动模糊、低分辨率等，以测试系统对这些因素的鲁棒性和容错能力。

非均匀分布:程序的输入空间、执行空间和输出空间往往不是均匀分布。例如，人脸识别系统中，不同面部特征（性别、年龄、肤色）可能具有不同的概率分布。



使用概率分布函数生成测试数据：

- 男性和女性面孔的概率分别为 0.4 和 0.6
- 年轻和老年面孔的概率分别为 0.3 和 0.7
- 浅色和深色皮肤的概率分别为 0.2 和 0.8

课后思考与练习：

在Triangle程序的随机测试中，需要定义三个输入参数，即三角形的三个边长。这三个输入参数a, b, c看起来是1-100的整数均匀分布。但无论是Triangle的程序执行空间还是输出空间，都是非均匀分布。思考一下如何更有效地进行随机测试生成。

在NextDay程序的随机测试中，更侧重考虑测试输入数据的边界情况。例如，需要测试程序是否能够正确地处理一些边界数据，例如输入的月份、日期和年份是否超出了合理的范围。可以使用随机数生成器来生成这些边界数据，并测试程序在处理这些数据时的表现。

在MeanVar程序的随机测试中，也需要考虑输入数据的分布情况。例如，还可以使用正态分布、指数分布、二项分布或泊松分布等常见的概率分布来生成数据。然后使用随机测试工具根据这个非均匀概率分布模拟用户行为，生成测试数据并调用方差程序。

课后思考与练习：

对于超市管理系统，可以从以下几个关键方面开展随机测试。首先是数据输入环节，超市里商品种类繁多，数据信息量大。操作流程方面，超市管理系统涉及进货、销售、库存管理等多个流程。我们可以随机打乱操作顺序，模拟一些不按常规流程来的操作等。

对于办公软件，随机测试首先从数据输入角度，考虑文字、表格、演示等多种输入类型，输入类型同时需要结合操作流程进行。随机测试多用户协作场景，观察软件如何处理操作和数据冲突。WPS和永中办公还有丰富的插件和扩展功能，这也是随机测试的范畴。

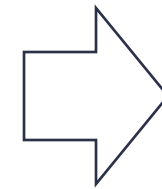
鸿蒙小艺作为智能软件，主要通过语音或文字接收用户指令，首先考虑从语言和文字的各自指令出发进行随机测试。使用鸿蒙小艺的家庭环境中，多个用户可能同时向小艺下达不同指令。鸿蒙小艺还集成各类第三方插件或服务，例如智能订餐还需随机模拟不同的地理位置。

回顾与讨论

随机不随意



- 多样化输入数据
- 多样性操作交互
- 多用户并发模拟



- 智能场景拓展
- 测试工具优化
- 自适应测试调整